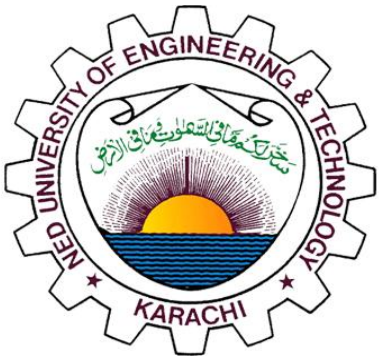
NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY



NETWORK AND INFORMATION SECURITY (CT-486)

Course Instructor: Dr. Shariq Mahmood Khan

GROUP MEMBERS

Fabiha Faheem (CR-23002)

Marium Habib (CR-23004)

Hareem Khan (CR-23023)

Emaan Rashid (CR-23029)

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NO
1	Executive Summary	3
2	Cipher Overview	3
3	Algorithms	4
4	How the Classical Techniques were Combined	5
5	Security Analysis	6
6	Attack Method and Results	7
7	Suggestions for Improving Security Without Large Added Complexity	8
8	Performance Metrices	9
9	Conclusion	11
10	Appendix	12

1. EXECUTIVE SUMMARY:

This project implements a classical hybrid cipher that applies a Vigenere stage followed by a 2x2 Hill stage. The whole secret is a 10-letter key: the first six letters form the Vigenere key and the last four fill the Hill matrix in row-major order. The program also contains a known-prefix attack, a descriptive frequency analyser and a benchmarking routine.

The two stages are linear or affine over the integers modulo 26, so the complete cipher collapses to C_j = H*P_j + t_(j mod 3) for every two-letter block j, where H is the Hill matrix and t takes only three secret values. This structure is the root of every attack. In our experiments the built-in known-prefix attack recovered 80 of 80 benchmark messages from a 20-letter prefix; a 12-letter chosen plaintext reveals the entire key; and a ciphertext-only attack that we implemented as an extension recovered the key and plaintext in 10 of 10 trials once the English message reached 200 letters, in about half a second. Encryption and decryption are fast (roughly 1.4 to 2.0 million letters per second).

Aspect	Result
Nominal key space	$26^{10} = 2^{47.0}$ raw; 48,576,387,944,448 valid keys = $2^{45.5}$
Known-prefix attack	Needs at least 10 known letters; 99.5% (random text) and 97.0% (English) success at 20 letters; 0.06 to 0.20 ms
Chosen-plaintext attack	12 chosen letters recover the whole key
Ciphertext-only attack	10/10 full recoveries at 200 or more English letters; 0.5 to 2.2 s
Encryption / decryption speed	About 1.4 to 2.0 million letters per second

2. CIPHER OVERVIEW

- **Preprocessing:** Plaintext is cleaned by discarding every character outside ASCII A-Z and converting lowercase to uppercase. Spaces, digits, punctuation and capitalisation are therefore lost and cannot be recovered after decryption.
- **Key:** The key must be exactly ten uppercase letters, for example CRYPTOHILL. Letters 1 to 6 (CRYPTO) are the repeating Vigenere key. Letters 7 to 10 (HILL) are read as numbers (H=7, I=8, L=11, L=11) and form the matrix [[7, 8], [11, 11]]. A key is rejected unless gcd(det, 26) = 1, because only then does the matrix have an inverse modulo 26.
- **Ciphertext format:** The ciphertext is one marker letter followed by an even-length body. The marker is A when the cleaned plaintext length is even and B when it is odd. For odd lengths a single X is appended after the Vigenere stage so that the Hill stage always receives complete pairs.

Because the padding is added after Vigenere and removed before inverse Vigenere, it never disturbs the key stream alignment.

3. ALGORITHMS

- **Notation**
  Letters are numbered A = 0 to Z = 25 and all arithmetic is modulo 26. The key is K = k0 k1 ... k9. The Vigenere key is v = (k0, ..., k5). The Hill matrix is H = [[a, b], [c, d]] = [[k6, k7], [k8, k9]], with determinant D = a × d – b × c mod 26. The inverse is $H^{-1} = D^{-1} \times$ [[d, -b], [-c, a]] mod 26, where D^{-1} is the modular inverse of D.

- **Encryption algorithm**

1. Validate the key (length 10, letters A-Z) and compute D. If gcd(D, 26) is not 1, reject the key.
2. Clean the plaintext to P = p0 p1 ... p(n-1). If n = 0, reject it.
3. Set the marker M = 'B' if n is odd, otherwise 'A'.
4. Vigenere stage: for each position i, compute u_i = (p_i + v_(i mod 6)) mod 26.
5. If M = 'B', append the pad letter X to u so that its length is even.
6. Hill stage: for each pair j (positions 2j and 2j+1), compute (c_2j, c_2j+1) = H * (u_2j, u_2j+1) mod 26.
7. Output M followed by the Hill output letters.

   Step 4 uses a repeating six-letter key, and step 6 treats the stream as independent column vectors of length two. The two stages are separate loops over the data, so the work is linear in n.

- **Decryption algorithm**

1. Validate the key and compute H^{-1} (this fails if the matrix is singular).
2. Validate the ciphertext: only A-Z, length at least 3, first letter A or B, and a body of even length. Nothing is silently cleaned.
3. Apply the inverse Hill stage to every pair of the body: (u_2j, u_2j+1) = H^{-1} * (c_2j, c_2j+1) mod 26.
4. If the marker is B, check that the last letter is X (otherwise report wrong key or ciphertext) and remove it.
5. Inverse Vigenere: p_i = (u_i - v_(i mod 6)) mod 26.
6. Output the cleaned plaintext.

   The order is the exact reverse of encryption, which is why padding removal precedes inverse Vigenere.

- **Worked example**

  Key “CRYPTOHILL”, plaintext “ATTACK AT DAWN TOMORROW”. The cleaned plaintext has 20 letters, so the marker is A. The Vigenere key is CRYPTO = (2, 17, 24, 15, 19, 14) and the Hill matrix is [[7, 8], [11, 11]], with D = 77 - 88 = -11 = 15 (mod 26), $D^{-1} = 7$ and H^{-1} = [[25, 22], [1, 23]].

Step	Block 0	Block 1	Block 2
Plaintext	A T	T A	C K
After Vigenere	C K	R P	V Y
Numeric	(2, 10)	(17, 15)	(21, 24)
After Hill	Q C	F O	B B

For block 0, 7×2 + 8×10 = 94 = 16 (Q) and 11×2 + 11×10 = 132 = 2 (C). Continuing through all ten blocks yields the ciphertext “AQCFOBBQCXUJUFAQNGJIH”. Decryption reverses this: for QC = (16, 2), 25×16 + 22×2 = 444 = 2 (C) and 1×16 + 23×2 = 62 = 10 (K), which restores CK, and subtracting CR from CK gives back AT.

4. HOW THE CLASSICAL TECHNIQUES WERE COMBINED

The cipher joins two ideas from classical cryptography that attack different weaknesses of a simple substitution cipher. The Vigenere cipher replaces the single Caesar shift by a sequence of six shifts, which flattens single-letter frequencies and hides the shape of the English distribution: the same plaintext letter is encrypted differently depending on its position modulo 6. Its weakness is that it does not mix neighbouring letters, so once the period is guessed each of the six columns is just a Caesar cipher. The Hill cipher has the opposite profile. Multiplying by an invertible matrix mixes the two letters of every pair, so a change in one plaintext letter alters two ciphertext letters (diffusion), and single-letter frequencies are further blurred because each output letter is a weighted sum of two inputs. Its weakness is that it is purely linear, so a handful of known plaintext blocks gives a solvable linear system. The Vigenere stage adds position dependence while the Hill stage adds local diffusion.

The order of the stages is Vigenere first and Hill second, and several design decisions follow from this. The Hill stage needs even-length input, so padding is added after Vigenere; a marker letter records whether padding was used, which makes decryption unambiguous without a length field. The period of the Vigenere key (6 letters) is a multiple of the Hill block size (2 letters), so the six shifts align exactly with three Hill blocks and the combined structure repeats every three blocks.

This alignment gives a compact algebraic description. Write the plaintext blocks as $P_j = (p_{2j}, p_{2j+1})$ and group the Vigenere key into the three pairs $w_0 = (k0, k1)$, $w_1 = (k2, k3)$, $w_2 = (k4, k5)$. Then the ciphertext block is $C_j = H(P_j + w_{(j \bmod 3)}) = H P_j + t_{(j \bmod 3)}$, where $t_m = H w_m$. The cipher is therefore a Hill cipher with an extra constant vector that cycles through only three values. It is an affine map within each phase, and the mapping between (w_0, w_1, w_2) and (t_0, t_1, t_2) is a bijection when H is invertible.

5. SECURITY ANALYSIS

- **Security strength**
  The Vigenère stage avoids a single fixed monoalphabetic substitution across the entire message. The Hill stage combines adjacent characters through a matrix operation, creating diffusion within each two-letter block. The key parser rejects malformed keys and non-invertible Hill matrices, preventing decryption from relying on a matrix inverse that does not exist. The ciphertext parser also validates the marker and body format rather than silently accepting arbitrary input.

- **Key space**
  A ten-letter key has $26^{10} = 1.41 \times 10^{14}$ values ($2^{47.0}$). Not all of them are valid, because the Hill matrix must be invertible. Exhaustive enumeration of all 26^4 matrices shows that exactly 157,248 are invertible modulo 26 (34.4%). The number of valid keys is therefore $26^6 * 157{,}248 = 48{,}576{,}387{,}944{,}448$, about 4.86×10^{13} or $2^{45.5}$. No two valid keys are equivalent, because (w, H) maps one-to-one to (t, H). For comparison, AES-128 offers 2^{128} keys. At a rate of 10^9 key trials per second (reasonable for optimised C or a GPU) a full search would take about 13.5 hours; at the roughly 3×10^4 to 7×10^4 trial decryptions per second that this Python code manages on a single core, it would take decades, but that figure is irrelevant because far cheaper attacks exist.

- **Linearity**
  The dominant weakness is the affine structure C_j = H*P_j + t_(j mod 3). Any known-plaintext, chosen-plaintext or statistical attack can exploit it, because the unknowns are only a 2x2 matrix (157,248 possibilities) and three translation vectors that are determined by the matrix once a single plaintext/ciphertext pair per phase is known. The effective search space after one known pair per phase is only 157,248 candidate matrices (about $2^{17.3}$), and linear algebra reduces even that to a direct computation.

- **Periodicity and determinism**
  The cipher is deterministic and has no initialisation vector or chaining. Identical plaintext at positions that are congruent modulo 6 yields identical ciphertext blocks, which is a structural leak similar to ECB mode in block ciphers. An eavesdropper can detect repeated words, repeated headers and boilerplate. The period is also very short: six letters is only three Hill blocks, so even short messages contain many samples of every phase.

- **Statistical properties**
  We measured the index of coincidence (IC) on 50 random keys and 3,000-letter windows of natural English (the bundled Python documentation, 327,702 letters). For uniformly random text IC = 1/26 = 0.0385, and for English it is about 0.066 to 0.070.

Text	IC	Largest single-letter frequency
English plaintext	0.0698	13.6%
Final ciphertext	0.0398	5.7%
Uniformly random text (theory)	0.0385	about 3.8%

The ciphertext is statistically close to random at the single-letter level, which means that simple frequency analysis, as performed by the frequency_comparison function, will not reveal the plaintext. This is the cipher's genuine strength.

- **Overall assessment**
  In short, the cipher resists naive frequency analysis but falls to attack that uses the algebra of the construction. Its security does not exceed that of a Hill cipher with 3 translation vectors.

6. ATTACK METHOD AND RESULTS

KNOWN-PREFIX ATTACK

Idea: The attacker knows the first L plaintext letters (for example a standard greeting or file header) and the complete ciphertext, but not the key. For two blocks i and j in the same phase, the translation t cancels: C_i - C_j = H(P_i - P_j). Two such difference pairs (dP, dC) whose dP vectors form an invertible matrix give $H = dC \times Dp^{-1}$. Once H is known, each phase offset follows from t_m = C - H P for one known block in that phase, and the Vigenere shifts come from w_m = H+-1 t_m.

Procedure: The attack is performed as following

1. Parses the ciphertext and cleans the prefix
2. Forms the full known blocks and sorts them into three phase groups
3. Stops with a reason if any phase has no known block
4. Computes all within-phase differences, stores them in a table keyed by dP (at most 26^2 entries), and flags an inconsistency if equal dP values map to different dC values
5. Searches for two stored differences with gcd(det, 26) = 1 and solves for H
6. Computes the three offsets
7. Verifies that the candidate explains every known pair
8. Decrypts the entire ciphertext and confirms that the result starts with the supplied prefix. Because the whole key is recovered, the whole plaintext is recovered, not just the prefix.

Requirements: At least one complete known pair in each of the three phases (6 letters) and two difference vectors with an invertible determinant. The smallest configuration that can satisfy both is five blocks, i.e., 10 known letters, with two blocks in phases 0 and 1 block in phase 2. A random pair of difference vectors is invertible with probability 157,248 / 456,976 = 34.4%, which predicts the success rate at 10 letters and explains why more letters raise the chance of finding a usable pair.

Results. We attacked messages of 200 letters with random valid keys, 400 trials per prefix length, using both uniform random text and natural English.

Known prefix (letters)	Success, random text	Success, English text
4, 6, 8	0%	0%
10	31.3%	35.8%
12	64.0%	60.8%
14	82.3%	75.8%
16	92.0%	91.3%
18	96.5%	94.0%
20	99.5%	97.0%
24	99.8%	97.8%
30	99.8%	100%
40	100%	100%

The measured 31% to 36% at 10 letters agrees closely with the theoretical 34.4%. Failures at 10 to 20 letters are almost always caused by difference vectors that are not invertible mod 26, not by a flaw in the cipher's protection. Natural English is slightly harder for this attack than random text at moderate prefix lengths because repeated letter pairs (such as TH or HE) produce repeated differences, but by 30 letters both reach essentially 100%. The program's built-in benchmark (seed 486, 20 trials per length, 20-letter prefix) reported 20/20 successes for plaintext lengths 41, 80, 161 and 320.

7. SUGGESTIONS FOR IMPROVING SECURITY WITHOUT LARGE ADDED COMPLEXITY

The following measures stay within classical building blocks and add a few lines to each stage rather than redesigning the cipher.

Improvement	Weakness addressed	Added complexity
Keyed non-linear substitution between the stages	Affine collapse; removes the linear algebra and ciphertext-only attacks	One 26-letter table derived from the key
Random IV and chaining (CBC-like)	Determinism, repeated blocks, the phase-difference trick	One extra block and one addition per block
3x3 Hill matrix and longer Vigenere key	Tiny matrix space; short period	Matrix loop width 3, longer key

Replace the parity marker with fixed padding	Length leak, known pad letter	A few lines
Use secrets for key generation, add a MAC	Predictable keys, malleability	Standard library calls

Add a non-linear layer: The decisive fix is to insert a key-dependent substitution (an S-box) between the Vigenere and Hill stages: a permutation of the 26 letters generated from a keyword or by a key-seeded shuffle. A substitution is not affine over Z26, so the composition no longer reduces to C = H*P + t, the difference trick no longer cancels the unknown offsets, and the matrix cannot be found by solving a linear system. The cost is one table lookup per letter and a table construction at start-up. Repeating substitution and Hill for two or three rounds with different subkeys gives a small substitution-permutation network that is still easy to implement and explain.

Add an initialisation vector and chaining. Prepend a random block to the ciphertext (replacing the parity marker) and add the previous ciphertext block to each plaintext block before the Hill stage. This ends determinism, so repeated plaintext no longer produces repeated ciphertext, and it breaks the same-phase differencing because t no longer repeats. By itself chaining does not remove linearity, so a more general linear-algebra attack still works with a few more known blocks, which is why it should be combined with the non-linear layer above.

Enlarge the algebraic structure. A 3x3 Hill matrix has far more candidates (roughly 26^9 before the invertibility condition) and needs three phases of 3x3 differences to attack, and a longer Vigenere key (for example 16 letters) lengthens the period beyond what short messages can expose. A period that is not a multiple of the block size also prevents the clean three-phase alignment. These changes are cheap, but on their own they raise the work factor only modestly because the system stays linear.

Fix format issues. Replace the parity marker with fixed padding such as PKCS-style padding of a random-looking pad, remove the known X, and generate keys with the secrets module. A message authentication code or a simple checksum placed inside the encrypted data would detect the malleability.

8. PERFORMANCE METRICS

All timings were measured with the program's own code on Python 3.12.3 in a container environment; absolute values depend on hardware and load and should be compared with each other, not with other systems.

- **Built-in benchmark**
  Seed 486, 20 trials per length, mean wall-clock milliseconds per trial, 20-letter known prefix.

Plain length	Cipher length	Attack success	Attack (ms)	Shift enc (ms)	Custom enc (ms)	Custom dec (ms)
41	43	20/20	0.0571	0.0071	0.0329	0.0289
80	81	20/20	0.0829	0.0143	0.0569	0.0557
161	163	20/20	0.1172	0.0248	0.0924	0.0903
320	321	20/20	0.1983	0.0450	0.1766	0.1828

The custom cipher takes about 4 to 5 times as long as a simple Caesar shift, which is expected because it performs two passes plus validation. Encryption and decryption times are almost identical, as the two directions use the same operations. Attack time is comparable with a single encryption.

- **Scaling with message size**

  Single runs with the key CRYPTOHILL, repeated for small sizes, 20-letter prefix for the attack.

Plaintext letters	Encrypt (ms)	Decrypt (ms)	Attack (ms)	Encrypt throughput (letters/s)
100	0.063	0.061	0.098	1.58 million
1,000	0.497	0.474	0.494	2.01 million
10,000	5.05	5.10	5.27	1.98 million
100,000	71.7	66.9	49.1	1.39 million
1,000,000	599	468	454	1.67 million

Times grow almost proportionally to message length, which confirms O(n) time for encryption and decryption (with O(n) memory for the string being built). The variation in throughput at the largest sizes reflects memory allocation and system noise in a single run.

- **Attack cost versus prefix length**

  For a 2,000-letter message the known-prefix attack took 0.94 ms with a 12-letter prefix and 1.31 ms with an 800-letter prefix. The pairwise search is bounded by at most 26^2 stored difference vectors, so it is $O(m^2 + n)$ in the number m of known blocks but with a fixed table size; most of the time is spent on the final O(n) decryption of the entire ciphertext. The ciphertext-only attack costs $O(|GL(2, Z26)| \times n) = O(157{,}248 \times n)$ operations, measured at 0.3 to 2.2 s for 60 to 1,000 letters using vectorised NumPy.

- **Complexity summary**

Operation	Time	Extra space
Encryption / decryption	O(n)	O(n)
Known-prefix attack	$O(m^2 + n)$, table of at most 676 entries	O(m + n)
Chosen-plaintext attack	O(1) after 12 letters	O(1)
Ciphertext-only attack	O(157,248 * n)	O(chunk * 156) in the vectorised version
Exhaustive search	$O(4.86 \times 10^{13} * k)$ for k test letters	O(1)

The cipher is cheap, which is desirable in itself, but the table shows an extreme imbalance: legitimate use costs about 0.5 microseconds per letter while breaking it costs less than a millisecond given a short prefix.

9. CONCLUSION

The Vigenere + Hill construction validates keys and ciphertext strictly, handles odd lengths cleanly with the marker and post-Vigenere padding, and round-trips every tested message. Its ciphertext also looks random at the single-letter level, with an index of coincidence of 0.0398 against 0.0385 for pure noise. Algebraically the cipher is a Hill cipher with three cycling translation vectors, so it is broken by a 10-letter known prefix, a 12-letter chosen plaintext, or about 200 letters of ciphertext alone, in each case within milliseconds or seconds. A keyed non-linear substitution, chaining with a random IV and a larger matrix would remove the attacks at modest cost.

APPENDIX: CODE

```
#!/usr/bin/env python3

from collections import Counter
from math import gcd
import random
import string
import time

ALPHABET = string.ascii_uppercase
# Approximate expected percentages in ordinary English prose.
ENGLISH = (8.17, 1.49, 2.78, 4.25, 12.70, 2.23, 2.02, 6.09, 6.97,
           0.15, 0.77, 4.03, 2.41, 6.75, 7.51, 1.93, 0.10, 5.99,
           6.33, 9.06, 2.76, 0.98, 2.36, 0.15, 1.97, 0.07)


def clean_plaintext(text):
    """Keep only ASCII letters and convert lowercase to uppercase."""
    return ''.join(c.upper() for c in text if 'A' <= c <= 'Z' or 'a' <= c <= 'z')


def number(letter):
    return ord(letter) - ord('A')


def letter(value):
    return chr(value % 26 + ord('A'))


def inverse_matrix(matrix):
    """Return the 2x2 matrix inverse mod 26, or raise ValueError."""
    (a, b), (c, d) = matrix
    det = (a * d - b * c) % 26
    if gcd(det, 26) != 1:
        raise ValueError('Hill matrix determinant must be coprime to 26.')
    inv_det = pow(det, -1, 26)
    return ((d * inv_det % 26, -b * inv_det % 26),
            (-c * inv_det % 26, a * inv_det % 26))


def matrix_times_vector(matrix, vector):
    (a, b), (c, d) = matrix
    x, y = vector
```

```
    return ((a*x + b*y) % 26, (c*x + d*y) % 26)


def matrix_multiply(left, right):
    return tuple(tuple(sum(left[i][k] * right[k][j] for k in range(2)) % 26
                       for j in range(2)) for i in range(2))


def parse_key(key):
    if len(key) != 10 or any(c not in ALPHABET for c in key):
        raise ValueError('Key must be exactly 10 uppercase ASCII letters A-Z.')
    h = [number(c) for c in key[6:]]
    hill = ((h[0], h[1]), (h[2], h[3]))
    inverse_matrix(hill)  # reject non-invertible matrices
    return key[:6], hill


def parse_ciphertext(ciphertext):
    """Strictly validate the new marker + body format; no silent cleaning."""
    if any(c not in ALPHABET for c in ciphertext):
        raise ValueError('Ciphertext must contain uppercase ASCII letters A-Z
only.')
    if len(ciphertext) < 3:
        raise ValueError('Ciphertext needs a marker and at least one two-letter
block.')
    if ciphertext[0] not in 'AB':
        raise ValueError('Marker must be A (even) or B (odd).')
    body = ciphertext[1:]
    if len(body) % 2:
        raise ValueError('The ciphertext body after the marker must have even
length.')
    return ciphertext[0], body


def vigenere_encrypt(text, key):
    return ''.join(letter(number(c) + number(key[i % 6])) for i, c in
enumerate(text))


def vigenere_decrypt(text, key):
    return ''.join(letter(number(c) - number(key[i % 6])) for i, c in
enumerate(text))


def hill_transform(text, matrix):
    """Transform an even-length text, with matrix or its inverse."""
```

```
    if len(text) % 2:
        raise ValueError('Hill input must contain complete pairs.')
    result = []
    for i in range(0, len(text), 2):
        x, y = matrix_times_vector(matrix, (number(text[i]), number(text[i+1])))
        result.extend((letter(x), letter(y)))
    return ''.join(result)


def encrypt(plaintext, key):
    vig, hill = parse_key(key)
    plain = clean_plaintext(plaintext)
    if not plain:
        raise ValueError('Plaintext must contain at least one ASCII letter.')
    marker = 'B' if len(plain) % 2 else 'A'
    intermediate = vigenere_encrypt(plain, vig)
    if marker == 'B':
        intermediate += 'X'  # padding belongs to the Hill input, not plaintext
    return marker + hill_transform(intermediate, hill)


def decrypt(ciphertext, key):
    vig, hill = parse_key(key)
    marker, body = parse_ciphertext(ciphertext)
    intermediate = hill_transform(body, inverse_matrix(hill))
    if marker == 'B':
        if intermediate[-1] != 'X':
            raise ValueError('Invalid odd-length padding (or wrong
key/ciphertext).')
        intermediate = intermediate[:-1]
    return vigenere_decrypt(intermediate, vig)


def known_plaintext_attack(known_prefix, ciphertext):

    marker, body = parse_ciphertext(ciphertext)
    prefix = clean_plaintext(known_prefix)
    plain_length = len(body) - (marker == 'B')
    if not prefix:
        raise ValueError('The known prefix must contain at least one ASCII
letter.')
    if len(prefix) >= plain_length:
        raise ValueError('The known prefix must be shorter than the message.')

    blocks = []
```

```
    groups = [[], [], []]
    # Ignore an incomplete prefix pair. The padded block is never a full
    # plaintext pair for an odd-length message, even if the prefix is long.
    for j in range(len(prefix) // 2):
        p = (number(prefix[2*j]), number(prefix[2*j+1]))
        c = (number(body[2*j]), number(body[2*j+1]))
        blocks.append((j % 3, p, c))
        groups[j % 3].append((p, c))
    if any(not group for group in groups):
        return {'success': False, 'reason': 'A phase is unobserved: need at least
one complete known pair in each of the three phases.'}

    differences = {}  # delta P -> corresponding delta C, bounded by 26^2
    hill = None
    for group in groups:
        for i in range(len(group)):
            p1, c1 = group[i]
            for j in range(i):
                p0, c0 = group[j]
                dp = ((p1[0]-p0[0]) % 26, (p1[1]-p0[1]) % 26)
                dc = ((c1[0]-c0[0]) % 26, (c1[1]-c0[1]) % 26)
                if dp in differences:
                    # Inconsistency is evidence that the prefix/ciphertext do
                    # not satisfy the assumed model, even if no basis is found.
                    if differences[dp] != dc:
                        return {'success': False, 'reason': 'Known pairs are
inconsistent with the cipher model.'}
                    continue
                for old_dp, old_dc in differences.items():
                    delta_p = ((old_dp[0], dp[0]), (old_dp[1], dp[1]))
                    if gcd(old_dp[0]*dp[1] - dp[0]*old_dp[1], 26) == 1:
                        delta_c = ((old_dc[0], dc[0]), (old_dc[1], dc[1]))
                        hill = matrix_multiply(delta_c, inverse_matrix(delta_p))
                        break
                if hill is not None:
                    break
                differences[dp] = dc
            if hill is not None:
                break
        if hill is not None:
            break
    if hill is None:
        return {'success': False, 'reason': 'No two same-phase difference vectors
have an invertible determinant mod 26. Try a longer or more varied prefix.'}
    try:
```

```
        inverse_hill = inverse_matrix(hill)
    except ValueError:
        return {'success': False, 'reason': 'Recovered Hill matrix is not
invertible; inputs are inconsistent.'}

    offsets = []
    for group in groups:
        p, c = group[0]
        hp = matrix_times_vector(hill, p)
        offsets.append(((c[0]-hp[0]) % 26, (c[1]-hp[1]) % 26))
    # A candidate must explain EVERY complete known pair, not just the basis.
    for phase, p, c in blocks:
        hp = matrix_times_vector(hill, p)
        predicted = ((hp[0]+offsets[phase][0]) % 26,
                     (hp[1]+offsets[phase][1]) % 26)
        if predicted != c:
            return {'success': False, 'reason': 'Recovered candidate is
inconsistent with the known plaintext pairs.'}

    # Recover equivalent Vigenere shifts from t = H*k, then reverse stages.
    shifts = [matrix_times_vector(inverse_hill, t) for t in offsets]
    vig = ''.join(letter(x) for pair in shifts for x in pair)
    intermediate = hill_transform(body, inverse_hill)
    if marker == 'B':
        if intermediate[-1] != 'X':
            return {'success': False, 'reason': 'Recovered candidate has invalid
odd-length Hill padding.'}
        intermediate = intermediate[:-1]
    recovered = vigenere_decrypt(intermediate, vig)
    # Also checks a trailing unpaired known character.
    if not recovered.startswith(prefix):
        return {'success': False, 'reason': 'Recovered candidate does not match
the entire supplied prefix.'}
    return {'success': True, 'plaintext': recovered, 'hill_matrix': hill,
            'vigenere_key': vig, 'offsets': offsets}


def frequency_comparison(ciphertext):
    """Print descriptive letter frequencies, excluding the parity marker."""
    _, body = parse_ciphertext(ciphertext)
    counts = Counter(body)
    print('\nLetter  Ciphertext %  Approx. English %')
    for i, c in enumerate(ALPHABET):
        print(f'  {c}       {counts[c] / len(body) *
100:6.2f}          {ENGLISH[i]:6.2f}')
```

```
    print('These frequencies are descriptive, not a break or proof of security.')
    print('Short texts and non-English/uniform random texts can differ greatly
from English.')


def random_key(rng):
    """Generate a valid key for reproducible experiments, not for security."""
    while True:
        key = ''.join(rng.choice(ALPHABET) for _ in range(10))
        try:
            parse_key(key)
            return key
        except ValueError:
            pass


def shift_encrypt(text, shift=3):
    return ''.join(letter(number(c) + shift) for c in text)


def benchmark(trials=20, seed=486):
    """Time real trials. Success means the entire recovered text equals truth."""
    rng = random.Random(seed)
    print('\nEXPERIMENT: reproducible uniform random A-Z text and valid random
keys')
    print(f'Seed: {seed}; trials per length: {trials}; known prefix: first 20
letters.')
    print('Lengths below are plaintext lengths; ciphertext has a marker and
possible pad.')
    print('Times are mean wall-clock milliseconds per trial, including
failures.')
    print('  Plain   Cipher  Success   Attack ms   Shift enc ms   Custom enc
ms   Custom dec ms')
    for length in (41, 80, 161, 320):
        successes = 0
        totals = [0.0, 0.0, 0.0, 0.0]
        ct_len = 0
        for _ in range(trials):
            message = ''.join(rng.choice(ALPHABET) for _ in range(length))
            key = random_key(rng)
            start = time.perf_counter()
            shift_encrypt(message)
            totals[1] += time.perf_counter() - start
            start = time.perf_counter()
            ciphertext = encrypt(message, key)
```

```
            totals[2] += time.perf_counter() - start
            start = time.perf_counter()
            roundtrip = decrypt(ciphertext, key)
            totals[3] += time.perf_counter() - start
            if roundtrip != message:
                raise RuntimeError('Encryption/decryption round trip failed.')
            start = time.perf_counter()
            result = known_plaintext_attack(message[:20], ciphertext)
            totals[0] += time.perf_counter() - start
            successes += int(result['success'] and result['plaintext'] ==
message)
            ct_len = len(ciphertext)
        ms = [x * 1000 / trials for x in totals]
        print(f'{length:7d} {ct_len:8d}  {successes:2d}/{trials:<2d} '
              f'{100*successes/trials:5.1f}%  {ms[0]:9.4f}   {ms[1]:12.4f}'
              f'   {ms[2]:13.4f}   {ms[3]:13.4f}')
    print('\nTimes depend on machine, system load and Python; fixed-seed data is
reproducible, timings are not.')
    print('Timing compares these Python implementations; custom encryption
includes validation and both stages.')
    print('Encryption, decryption and Shift are O(n); attack is O(m^2 + n) for m
known pairs, fixed alphabet.')
    print('These random-text trials do not predict success on natural English or
other datasets.')


def main():
    print('VIGENERE + HILL: educational demonstration, not secure encryption.')
    print('FORMAT CHANGE: ciphertext starts with A (even) or B (odd), unlike the
original.')
    print('Plaintext is cleaned to ASCII A-Z; spaces, punctuation and formatting
are lost.')
    while True:
        print('\n1. Encrypt  2. Decrypt  3. Known-prefix attack  4. Benchmark  5.
Exit')
        try:
            choice = input('Choose 1-5: ').strip()
            if choice == '1':
                key = input('10-letter uppercase key (example CRYPTOHILL):
').strip()
                plaintext = input('Plaintext: ')
                print('Cleaned plaintext:', clean_plaintext(plaintext))
                print('Ciphertext:', encrypt(plaintext, key))
            elif choice == '2':
                key = input('10-letter uppercase key: ').strip()
```

```
                ciphertext = input('Marked uppercase ciphertext: ').strip()
                print('Recovered cleaned plaintext:', decrypt(ciphertext, key))
            elif choice == '3':
                ciphertext = input('Full marked uppercase ciphertext: ').strip()
                prefix = input('Known plaintext PREFIX (not the whole message):
')
                print('\nThe attacker uses only this prefix and the ciphertext,
not the secret key.')
                frequency_comparison(ciphertext)
                result = known_plaintext_attack(prefix, ciphertext)
                if result['success']:
                    print('\nRecovered plaintext:', result['plaintext'])
                    print('Recovered Hill matrix:', result['hill_matrix'])
                    print('Recovered Vigenere shifts as letters:',
result['vigenere_key'])
                    print('Candidate matches all known letters; unknown text
cannot be independently verified without ground truth.')
                else:
                    print('\nAttack could not recover the message:',
result['reason'])
                print('Requires the correct aligned prefix, all three phases, and
an invertible pair of differences;')
                print('this particular method can fail even when other
cryptanalysis may succeed.')
            elif choice == '4':
                benchmark()
            elif choice == '5':
                print('Goodbye.')
                break
            else:
                print('Please choose a number from 1 to 5.')
        except ValueError as exc:
            print('Input error:', exc)
        except (EOFError, KeyboardInterrupt):
            print('\nGoodbye.')
            break


if __name__ == '__main__':
    main()
```